

Pipeline de données temps réel sur la mobilité urbaine rennaise

Document de cadrage du projet

Wanda NANDA — INSA Rennes, Informatique spécialité IA

Projet personnel encadré — Data Engineering

En une phrase. Construire une chaîne de traitement de données complète qui ingère en continu la position temps réel des bus du réseau STAR de Rennes, la confronte à l'offre théorique de transport, et produit des indicateurs de ponctualité exploitables — le tout industrialisé avec les outils standards du métier de data engineer.

1. Pourquoi ce projet

1.1. Objectif de carrière

Ce projet répond à un manque précis dans le profil actuel : tous les travaux data réalisés jusqu'ici sont **mono-machine et en traitement par lots**. Or les offres de stage et d'emploi en data engineering, en particulier chez les opérateurs télécom, exigent systématiquement quatre familles de compétences absentes du CV :

- le **traitement distribué** (Spark, Databricks) ;
- le **traitement en flux** (Kafka, streaming) ;
- l'**orchestration** de traitements planifiés (Airflow, dbt) ;
- la **qualité et l'observabilité** des données en production.

Le pari de ce projet est qu'un seul projet ambitieux et mené jusqu'à l'industrialisation vaut mieux que trois petits projets tutoriels. Un pipeline ETL sur le jeu de données *NYC Taxi* est vu plusieurs centaines de fois par an par un recruteur data. Un pipeline streaming sur une source publique française, avec gestion des données en retard et tests de qualité automatisés, ne l'est pas.

1.2. Objectif pédagogique

Le projet est structuré comme un module d'enseignement : cours théorique, travaux pratiques guidés, puis application directe au projet. L'objectif n'est pas de faire fonctionner un empilement d'outils, mais de comprendre **pourquoi** chaque brique existe et quel problème elle résout. À la fin, il faut être capable de justifier chaque choix d'architecture devant un ingénieur expérimenté.

1.3. Objectif technique

Livrer un dépôt Git autonome que n'importe qui peut cloner et démarrer en moins de dix minutes, produisant automatiquement des indicateurs de ponctualité du réseau STAR à partir des flux publics.

2. Le domaine : comprendre la donnée de transport

2.1. Deux sources, deux natures

Le réseau STAR (exploité par Keolis pour Rennes Métropole) publie ses données en open data selon deux standards internationaux complémentaires.

	GTFS (statique)	GTFS-RT (temps réel)
Contenu	L'offre <i>théorique</i> : lignes, arrêts, courses, horaires prévus	L'état <i>réel</i> : position des véhicules, retards constatés, perturbations
Format	Archive ZIP de fichiers CSV	Flux binaire Protocol Buffers
Fréquence	Une publication par nuit	Rafraîchissement de l'ordre de la dizaine de secondes
Volume	Quelques dizaines de Mo	Quelques centaines de ko par appel
Rôle dans le projet	Référentiel de comparaison	Flux d'événements à ingérer

Le format **GTFS** est le standard mondial de description d'une offre de transport public : il est utilisé par Google Maps, Citymapper et l'ensemble des calculateurs d'itinéraires. Le format **GTFS-RT** en est l'extension temps réel, encodée en Protocol Buffers — un format binaire à schéma défini, plus compact et plus contraint que du JSON.

2.2. La problématique centrale

Le retard n'existe dans aucune des deux sources. Le GTFS dit ce qui *devait* arriver. Le GTFS-RT dit où sont les bus *maintenant*. L'écart entre les deux — la ponctualité réelle du réseau — doit être *calculé* en confrontant les deux univers.

C'est précisément la valeur ajoutée d'un data engineer : produire une métrique métier qui n'est présente dans aucun système source, de façon fiable, reproductible et automatisée. Toute la difficulté du projet découle de cette phrase.

3. Les problématiques d'ingénierie

Ce sont les vraies difficultés du projet. Chacune fait l'objet d'une ou plusieurs séances du module.

3.1. P1 — Un flux non borné

En traitement par lots, on connaît la taille du jeu de données, on peut le relire, on sait quand le traitement se termine. Ici, rien de tout cela : le flux ne s'arrête jamais, on ne peut pas tout garder en mémoire, et une interruption ne doit pas faire perdre la position de lecture. Cela impose un intermédiaire capable de *tamponner* et de *rejouer* les événements : c'est le rôle de Kafka.

3.2. P2 — Temps de l'événement contre temps du traitement

Un bus émet sa position à 14h03. Le pipeline la reçoit à 14h05 à cause de la latence réseau. Si les agrégations sont faites sur l'heure de réception, les statistiques deviennent fausses dès que le réseau hoquette. Toutes les métriques métier doivent être calculées sur le **temps de l'événement**, ce qui oblige à gérer les messages arrivant en retard, et donc à décider combien de temps on accepte d'attendre un retardataire avant de figer un résultat. C'est la notion de *watermark*.

3.3. P3 — Un référentiel qui change toutes les nuits

Le GTFS est republié chaque nuit, avec des identifiants de course qui peuvent changer. Rejouer les données du mardi avec le référentiel du jeudi produit des jointures fausses ou vides. Il faut donc **versionner le référentiel** et garantir que chaque événement est joint à la version qui était en vigueur au moment où il s'est produit. C'est un problème classique de gestion de données de référence.

3.4. P4 — Idempotence et replay

Un traitement rejoué deux fois doit produire exactement le même résultat, pas des doublons. Un incident survenu à 3h du matin doit pouvoir être rattrapé le lendemain sans intervention manuelle. Cette exigence conditionne la façon d'écrire les fichiers, de nommer les partitions et d'écrire les

tâches d'orchestration.

3.5. P5 — Qualité et confiance

Une donnée temps réel est sale par nature : coordonnées GPS aberrantes, véhicules qui disparaissent du flux, horodatages incohérents, champs optionnels vides. Sans contrôles automatisés, le pipeline produit silencieusement des indicateurs faux, ce qui est pire que de ne rien produire. Il faut définir des règles explicites et les tester à chaque exécution.

3.6. P6 — Volumétrie et organisation du stockage

À raison d'un appel toutes les vingt secondes sur plusieurs centaines de véhicules, l'historique croît en continu. La façon dont les fichiers sont découpés et partitionnés détermine si une requête portant sur une journée lit dix mégaoctets ou dix gigaoctets. C'est le sujet du partitionnement et du format colonne.

4. Les questions métier auxquelles le pipeline répondra

Le pipeline n'est pas une fin en soi : il doit répondre à des questions formulées à l'avance. Elles pilotent la modélisation de la couche analytique.

1. Quel est le retard moyen et médian par ligne, par tranche horaire et par jour de la semaine ?
2. Quelles lignes se dégradent le plus aux heures de pointe par rapport aux heures creuses ?
3. Quels arrêts concentrent le plus de retard, et à quels moments ?
4. Combien de véhicules sont en service à un instant donné, et comment cela se compare-t-il à l'offre théorique prévue ?
5. Peut-on détecter en quasi temps réel une anomalie de service — une ligne qui décroche brutalement par rapport à son comportement habituel ?

La question 5 est celle qui donne le plus de valeur au projet : elle transforme un outil de reporting en outil de supervision, ce qui est exactement l'usage industriel d'un pipeline temps réel.

5. Architecture cible

Le pipeline est organisé en couches, selon le modèle dit **médaille**. Le principe non négociable est que la donnée brute n'est jamais écrasée : elle est le seul point de vérité qui permet de rejouer l'histoire quand la logique métier s'avère fautive.

Étage	Technologie	Rôle
Sources	API HTTP publiques	Flux GTFS-RT binaire et archive GTFS statique
Ingestion	Producteur Python	Interrogation périodique, décodage Protobuf, publication d'événements
Bus d'événements	Apache Kafka	Tampon durable, découplage producteur / consommateur, rejeu possible
Traitement flux	Spark Structured Streaming	Nettoyage, enrichissement, agrégations fenêtrées en temps de l'événement
Couche bronze	Fichiers Parquet	Événements bruts, jamais modifiés, partitionnés par date
Couche silver	Fichiers Parquet	Données typées, dédoublées, jointes au référentiel GTFS versionné
Couche gold	PostgreSQL	Tables d'indicateurs agrégés prêts à être requêtés
Transformation	dbt	Modèles SQL versionnés, documentés et testés
Orchestration	Apache Airflow	Planification, dépendances, reprise sur incident, rattrapage
Restitution	Dashboard	Visualisation des indicateurs de ponctualité
Industrialisation	Docker Compose, GitLab CI	Environnement reproductible, tests automatisés

Le flux temps réel et les traitements par lots coexistent : le streaming alimente le bronze en continu et calcule les indicateurs de supervision à faible latence, tandis que des traitements planifiés recalculent chaque nuit les agrégats historiques de façon fiable. Le choix entre une architecture dite *lambda* (deux chaînes distinctes) et *kappa* (une seule chaîne, le batch n'étant qu'un rejeu du flux) fait partie des décisions à argumenter dans le document d'architecture du projet.

6. Justification des choix technologiques

Chaque brique doit pouvoir être défendue en entretien. Le tableau ci-dessous donne la logique de sélection.

Techno	Pourquoi celle-ci	Alternative écartée
Kafka	Standard de fait du bus d'événements ; apporte la durabilité, le rejeu et le découplage. Compétence explicitement demandée dans les offres data télécom.	RabbitMQ (orienté messages, pas conçu pour le rejeu d'historique)
Spark	Moteur distribué le plus répandu en entreprise ; une API unique pour le flux et le lot. Passerelle directe vers Data-bricks.	Flink (excellent en streaming mais moins demandé en stage)
Parquet	Format colonne compressé, lecture sélective des colonnes, standard de tout lac de données.	CSV (aucune typage, aucune compression, illisible à grande échelle)
PostgreSQL	Déjà maîtrisé, suffisant pour la couche de restitution, coût nul en local.	Entrepôt cloud, hors budget pour un projet personnel
dbt	Introduit la rigueur logicielle dans le SQL : versionnement, tests, documentation, lignage. Très demandé.	Scripts SQL manuels, non testables
Airflow	Référence de l'orchestration ; expose les notions de dépendance, d'idempotence et de rattrapage.	Cron, qui ne gère ni dépendances ni reprise
Docker	Déjà maîtrisé ; garantit la reproductibilité de l'environnement.	Installation manuelle, non reproductible

7. Périmètre du projet

7.1. Dans le périmètre

- Ingestion continue des positions de véhicules et des mises à jour de courses.
- Récupération quotidienne et versionnement du référentiel GTFS.
- Calcul du retard par course et par arrêt, et agrégations associées.
- Détection d'anomalies de service en quasi temps réel.
- Tests de qualité de données automatisés et bloquants.
- Orchestration, conteneurisation et intégration continue.
- Documentation complète permettant la reprise du projet par un tiers.

7.2. Hors périmètre

- Le déploiement sur une plateforme cloud payante. L'architecture sera néanmoins conçue pour être portable, et le chemin de migration documenté.

- La haute disponibilité réelle : un seul broker Kafka, pas de réplication. Les mécanismes seront compris théoriquement.
- Toute prédiction de retard par apprentissage automatique. C'est une extension possible une fois le socle stable, pas un objectif initial.
- Une interface utilisateur soignée. Le dashboard sert à démontrer la chaîne, pas à être un produit.

8. Déroulé du module

Séance	Thème	Compétence visée
1	Cadrage et exploration de la source	Concevoir une architecture, lire une spécification, caractériser une source
2	Ingestion et producteur	Consommation d'API, décodage Protobuf, résilience réseau
3	Kafka : fondamentaux	Topics, partitions, offsets, groupes de consommateurs, garanties de livraison
4	Spark : modèle d'exécution	DataFrame, évaluation paresseuse, partitions, coût du <i>shuffle</i>
5	Spark Structured Streaming	Temps de l'événement, watermark, agrégations à état, points de reprise
6	Stockage et modélisation	Parquet, partitionnement, couches bronze / silver / gold
7	Orchestration et dbt	Graphes de tâches, idempotence, rattrapage, tests de modèles
8	Qualité, supervision, CI/CD	Contrats de données, tests bloquants, industrialisation
9	Valorisation	Dashboard, README, argumentaire d'entretien

Méthode. Chaque séance suit le même schéma : un cours théorique, une liste de notions à approfondir en autonomie, puis un travail pratique appliqué directement au projet. Les solutions ne sont pas fournies : en cas de blocage, le raisonnement et les tentatives doivent être présentés avant d'obtenir des indices. Chaque séance produit un livrable versionné sur Git, suivi d'une revue de code.

9. Livrables et critères d'évaluation

9.1. Livrable final

Un dépôt Git public contenant le code du pipeline, les fichiers d'infrastructure, les tests, la documentation d'architecture et un **README** permettant un démarrage complet en moins de dix minutes.

9.2. Critères de qualité

- **Reproductibilité** : un tiers clone le dépôt et obtient un pipeline fonctionnel sans intervention manuelle.
- **Robustesse** : le pipeline survit à une coupure réseau, à un redémarrage et à un rejeu.
- **Justifiabilité** : chaque choix d'architecture est documenté et argumenté.
- **Testabilité** : les transformations sont couvertes par des tests, exécutés automatiquement en intégration continue.
- **Lisibilité** : la structure du dépôt est compréhensible en cinq minutes.

10. Transposition professionnelle

L'argument d'entretien. Un flux de positions de véhicules géolocalisés, horodatés, émis en continu et comparés à une référence contractuelle est structuellement identique à un flux de télémétrie d'équipements réseau. Des sondes émettent des mesures, le pipeline les ingère, les confronte à un niveau de service attendu, et détecte les écarts. Le vocabulaire métier change, l'architecture ne change pas.

Les compétences produites par ce projet se traduisent directement en mots-clés attendus dans les offres de stage data : traitement distribué, streaming, orchestration, modélisation en couches, qualité de données, industrialisation. Elles se traduisent surtout en *récits* : comment une donnée en retard a été gérée, pourquoi un partitionnement a été choisi, comment un incident a été rattrapé. C’est ce type de récit, et non la liste des technologies, qui distingue un candidat en entretien technique.

Annexe — Sources de données

Ressource	Accès
GTFS statique (version courante)	https://eu.ftp.opendatasoft.com/star/gtfs/GTFS_STAR_BUS_METRO_EN_COURS.zip
GTFS-RT positions véhicules	https://proxy.transport.data.gouv.fr/resource/star-rennes-integration-gtfs-rt-vehicle-position
GTFS-RT mises à jour de courses	https://proxy.transport.data.gouv.fr/resource/star-rennes-integration-gtfs-rt-trip-update
Portail open data	https://data.rennesmetropole.fr

L’ancien jeu de données REST « position des bus en circulation » est en cours de dépréciation. L’exploitant recommande explicitement l’usage des flux GTFS-RT, dont le rafraîchissement est meilleur et qui sont synchronisés avec le GTFS en vigueur. Le projet attaque donc directement la source recommandée.

Précaution d’usage. Ces services sont publics et gratuits. Les appels doivent être espacés raisonnablement et les réponses brutes sauvegardées en local pour les phases de développement, afin de ne pas solliciter inutilement l’infrastructure.